

Machine Learning Framework for Snapdragon S7/S7+ Gen 1 Sound Platform Quick Start Guide

80-62823-1 AH

November 19, 2025

Confidential - Qualcomm Technologies, Inc. and/or its affiliated companies - May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to:
DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Contents

1	Qualcomm Machine Learning overview	3
2	Machine Learning workflow	5
2.1	Model training	6
2.2	Model quantization	7
2.3	Model conversion	7
2.4	Model inference	8
2.5	Model deployment	8
3	eAI tools for model conversion	9
3.1	eAI supported operations and eNPU3 offload	10
3.2	eAI design guidelines	11
4	Offloading operations using embedded Neural Processing Unit	34
4.1	eNPU data flow	37
5	Machine Learning runtime interface and APIs	39
6	Kymera Machine Learning capability	43
6.1	Machine Learning base capability	43
6.2	Machine Learning engine capability	44
6.3	Kymera Simulation Environment validation	49
7	Kymera Machine Learning example application	60
7.1	Setting up Machine Learning example application	60
7.2	Machine Learning example application	62
7.3	Debugging Machine Learning application using Kymera Debugging and Analysis Tool	65
7.4	Developing a new Machine Learning application	69
8	Working example: Creating and deploying Machine Learning model on target audio platform	70
8.1	Convert the model to eAI	70
8.2	Setup the Xtenso SDK and HiFi package	72
8.3	Prepare the Machine Learning application for the board	73
8.4	Erase the board	75

8.5	Build and burn the image to the device	76
8.6	Connect KDAT to the device	76
9	Revision history	77
10	Document references	80
11	Terms and definitions	81

Qualcomm
 Confidential - May Contain Trade Secrets
 2026-05-12 06:35:39 GMT
 hyungchul.won@lge.com

1 Qualcomm Machine Learning overview

The ADK and ADK Toolkit for Snapdragon S7/S7+ Gen 1 Sound Platform includes tools, software, and documentation to develop, optimize, and validate models under simulation before deploying them to the supported audio platforms.

The supported audio platforms are:

- QCC522x
- QCC722x

The Qualcomm Machine Learning ([ML](#)) framework enables ML model developers to deploy their ML models on the supported target audio platforms.

The S7/S7+ Gen 1 Sound Platform includes an AI runtime designed for embedded devices (eAI), which runs within the Kymera Audio Framework on its main DSP processor. It also includes a third generation AI accelerator called an embedded Neural Processing Unit ([eNPU3](#)) that is dedicated to accelerating the inference with low power.

Once the models are pre-trained, it must go through:

- eAI-aware quantization
- ONNX model exporting or rectifying
- eAI model conversion
- Verification

The Linux ADK Toolkit includes tools for model conversion, quantization, and validation of the model on x86 and HiFi4 simulation. If ML models are already quantized in TensorFlow Lite format, then they can be directly converted to eAI through the eAI Builder. After eAI models are verified with expected performance and accuracy, they are ready for integration and deployment on the target audio platforms.

The ADK provides interfaces and templates for developing a Kymera Audio Framework capability that allows model inference on the target audio platforms. The ML capability allows inferring the models on supported target platforms simulation and hardware. It can be integrated with a Kymera audio graph and application for any particular use case on the supported target audio platforms. For information about developing audio capabilities, see the *Kymera Capability Development User Guide*.

Note: You can download the document as a PDF, but it may contain formatting issues. For the best experience, refer to the HTML view.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyunchul.won@lge.com

2 Machine Learning workflow

The following figure shows the Machine Learning workflow and the different components involved in model training, quantization, conversion, and verification.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyunchul.won@lge.com

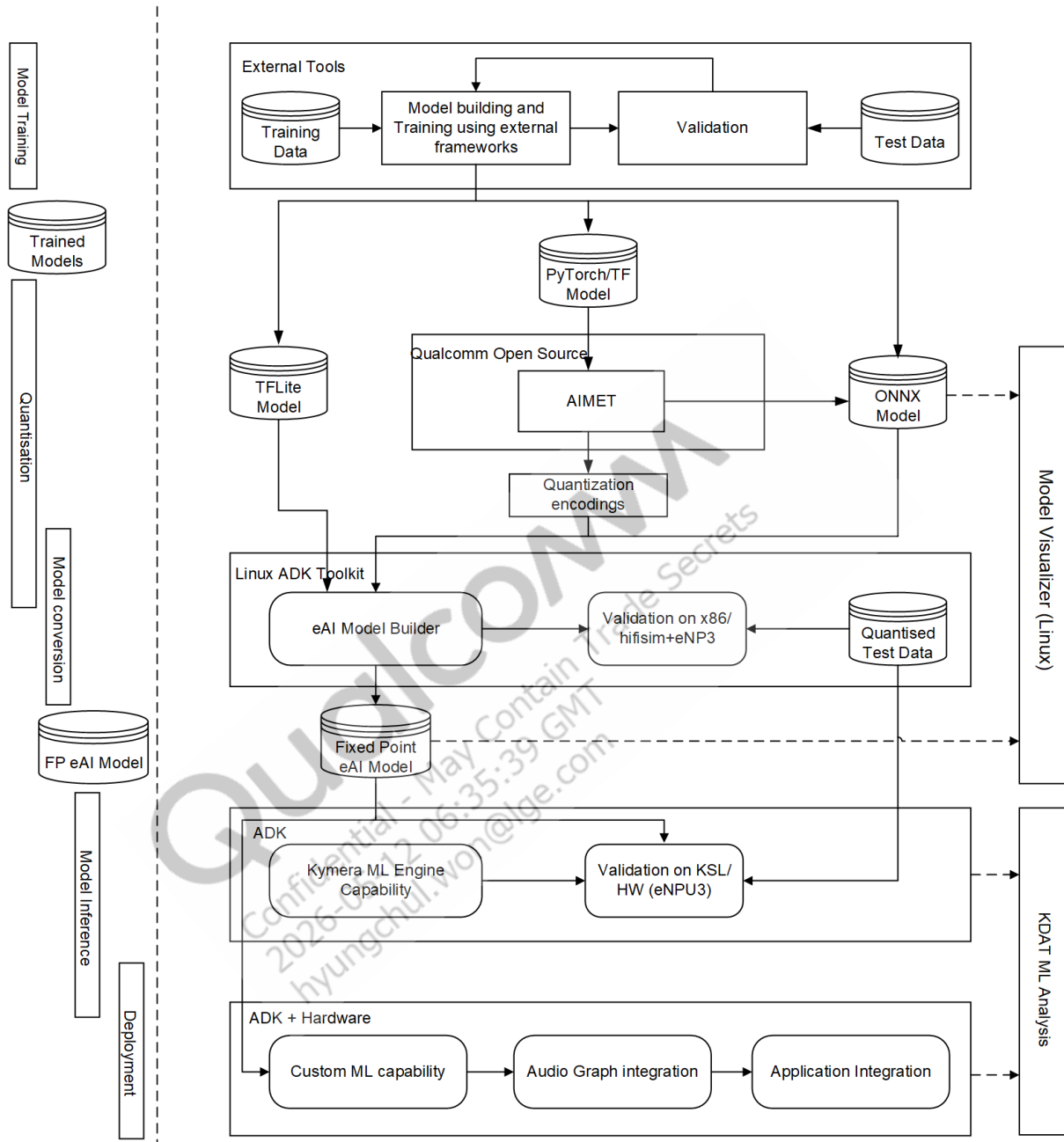


Figure1 Machine Learning workflow

2.1 Model training

To perform data collection and model training, use an external framework such as Pytorch or TensorFlow.

2.2 Model quantization

AI Model Efficiency Toolkit ([AIMET](#)) is a software toolkit that provides advanced quantization and compression techniques for trained neural network models.

AIMET supports PyTorch, TensorFlow, Keras and ONNX models. For more information about AIMET, see *GitHub - quic/aimet*: <<https://github.com/quic/aimet>>.

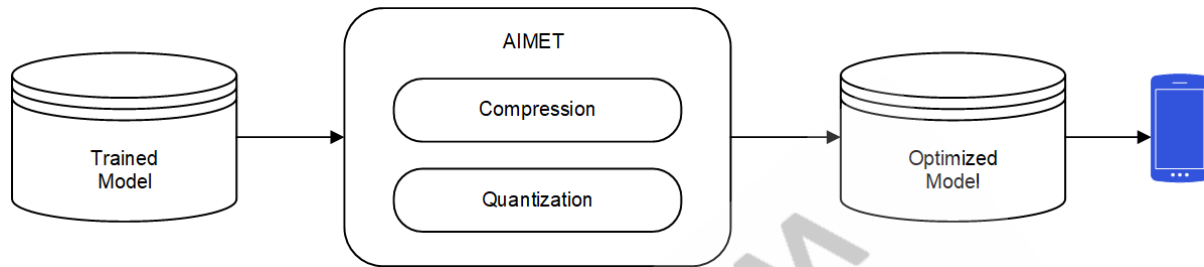


Figure2 Machine Learning model quantization

AIMET quantization works by adding simulation nodes to evaluate the expected effects of quantization on inputs, weights, and activations. The output of the process is a quantization encoding file in JSON format. This encoding file should be passed to the eAI model conversion tool, `eai_builder`, along with the floating point model in either ONNX or TensorFlow Lite format.

AIMET provides built-in facilities to convert a PyTorch model to ONNX format. This preserves the names of operator nodes and is therefore compatible with the generated quantization encoding file.

AIMET can be used to generate quantization encoding for a TensorFlow model. However, the TensorFlow saved model must be converted separately into an ONNX or TensorFlow Lite model. This can be done programmatically, or by using a tool such as `tflite_convert` (provided in TensorFlow 2.0 and later). Because the conversion process renames nodes in the model, the developer must edit the names of nodes in the generated quantization encoding file to match.

An alternative path for TensorFlow users is to convert the model to TensorFlow Lite format and use TensorFlow Lite's integer quantization API. In this case, the quantized TensorFlow Lite model can be passed directly to the `eai_builder` tool without specifying quantization encodings.

2.3 Model conversion

Before making model inference on S7 Gen 1 platform, the model must be converted to fixed point eAI format.

The eAI tools included in the ADK Toolkit can convert a trained ONNX/TensorFlow Lite model format to an eAI model, and verify it on an x86 or S7 Gen 1 HiFi simulator.

The pre-trained PyTorch and TensorFlow models must go through:

- eAI-aware quantization

- conversion to ONNX or TensorFlow Lite format. ONNX models may need to be passed through the ONNX rectifier tool
- eAI model conversion through `eai_builder`
- Verification

After eAI models are verified with expected performance and accuracy, they are ready to be integrated with the Kymera framework and deploy on the S7 Gen 1 Platform.

2.4 Model inference

The Kymera framework provides ML Engine capability that wraps the eAI runtime.

The ML engine capability is a test or reference capability that demonstrates model inference using the Kymera framework. The model inference can be done on the Kymera Simulation Environment ([KSL](#)) or on the audio subsystem of the S7 Gen 1 platform using the Kymera ML engine capability.

2.5 Model deployment

After the model is validated using the ML engine capability on KSL or hardware, it can be integrated into an existing capability or a new custom capability with required pre or post-processing to handle the activation input tensors and output tensors.

An ML base capability template is included in the ADK as a reference for implementing an ML capability. After integration, the capability can be integrated with a Kymera audio graph and the application for deployment on the hardware.

3 eAI tools for model conversion

The eAI tools included in the ADK Toolkit can convert a trained ONNX/TFLite model format to an eAI model and verify it on an x86 or S7 Gen 1 platform HiFi simulator.

The Linux ADK Toolkit includes the following tools under `<ADK_TOOLKIT_ROOT>/eai_runtime/tools` folder.

The following figure shows the eAI model visualizer, which is a viewer for neural network, deep learning, and machine learning models.

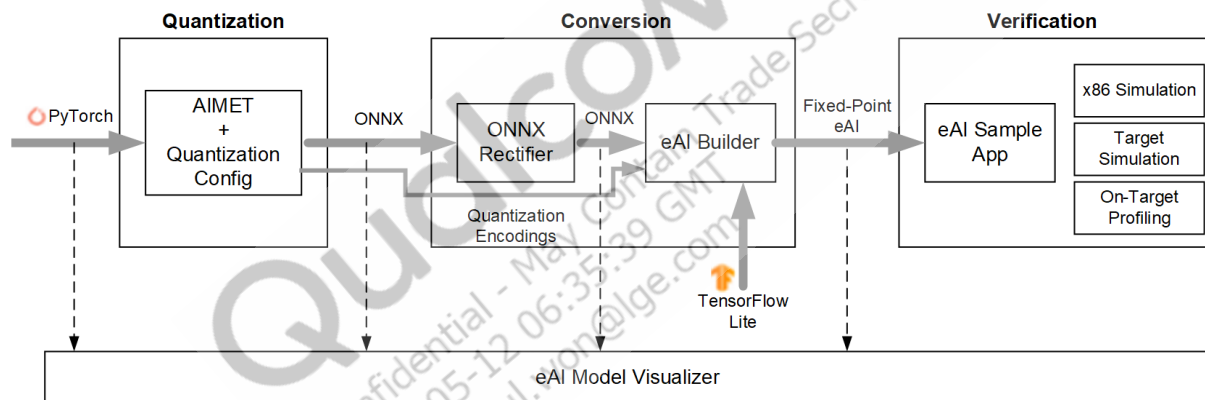


Figure1 eAI model visualizer

For detailed information with examples, see the online documentation under `<ADK_TOOLKIT_ROOT>/eai_runtime/docs` folder. The online documentation includes examples of creating the model and offline validation of models on the x86 Linux platform.

eAI builder

This eAI model builder converts an ONNX/TFLite model to an eAI Fixed point model. For more information, see [Working example: Creating and deploying Machine Learning model on target audio platform](#).

ONNX and TFLite floating point to eAI fixed point requires an accompanying JSON file that outlines quantization information.

ONNX model rectifier

To simplify the ONNX model, fold the constants, rename the nodes, and perform shape inference

offline.

Before feeding ONNX models into eAI model builder, models must be preprocessed or cleaned up. This script performs cleanup operations. The script can be modified for new operations required by the ONNX models.

eAI quantize

eAI Quantize can quantize or dequantize tensors for a given eAI model, using the quantization information stored in the model as default.

In general, quantizing/dequantizing tensor information is done so that comparisons can be made between fixed point or floating point data. One use case would be generating the appropriate input for a fixed/float eAI model before evaluation (for example, quantizing 32-bit float input to generate 8-bit fixed point input for a fixed eAI model).

Another use case would be quantizing/dequantizing model output so that comparisons can be made between the eAI output and original ONNX golden data (for example, dequantizing fixed point eAI output to float format, so the output can be compared with expected golden output).

eAI model visualizer

The Netronix Visualizer is a viewer for neural network, deep learning, and machine learning models.

It is based on the Open Source project from <https://github.com/lutzroeder/netron> and has been expanded to support eAI model definitions.

eAI sample app

eAI Sample App is the offline model runner application that runs the model on eAI runtime for validation.

Models created for x86 can run directly on Linux. Models created for Tensilica (with or without eNPU offloading) can be run using the S7 Gen 1 platform HiFi Simulator to validate the model.

3.1 eAI supported operations and eNPU3 offload

The following tables lists the supported eAI and eNPU3 offload for the release 2.x:

Supported hardware accelerated operations on eNPU3			
Add	Convolution (including DW-conv)	GRU	Quantize
Average Pool	Fully Connected	LSTM	ReLU
Batch Normalize	GEMM	MatMul	Sigmoid
Clip	Global AveragePool	Max Pool	Tanh

The following table lists the other supported operations:

Other supported operations			
Arg Max	Exp	Mul	Slice
Arg Min	Framer	No Op	Softmax
Concat	Gather	Pow	Sqrt
Constant	Inner Product	Reduce	Squeeze
Constant Fill	Log	Reduce Mean	Sub
Conv 1D	Log Softmax	Reshape	Transpose
Convolution Transpose	Max	RNN	UnSqueeze
Div	Max Out	Shape	Min
Pad	PReLU	RNN	Split

Note: Full list of supported operations can be found in the eAI header file `eai.h` and mentioned in `/eai_runtime/docs/model_builder/op_support.html` and `/eai_runtime/docs/model_builder/layer_offload.html`.

3.2 eAI design guidelines

Tensor notations

The following notations are used for representing tensors during eNPU3-based inference processing:

Definitions	Notation
Input image width	X_i
Input image height	Y_i
Number of input channels	C_i
Weight filter width	K_x
Weight filter height	K_y
Number of input channel weights	K_c
Number of Kernels	K_o
Output image width	X_o
Output image height	Y_o
Number of output channels	C_o

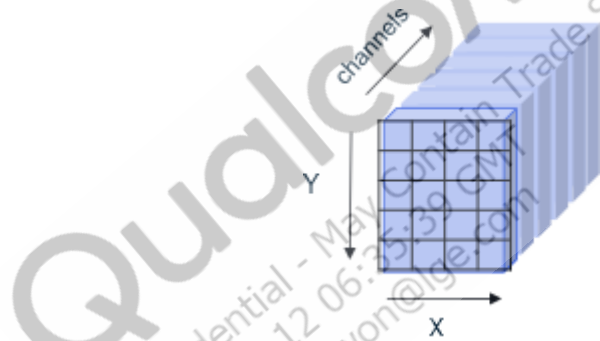


Figure2 Tensor notations

Note: Convolution activations are stored in channel-first format.

Bit-width support

eNPU3 supports the following bit-width combinations:

Weight bit-width	Activation bit-width	Output bit-width
8	8 or 16	16 or 8
16	16	16 or 8

Software-based support for alternative bit-widths may exist, depending on the specific operator and the target processor. The eNPU3 internally generates 32-bit output for many operations and subsequently quantizes them to 8-bit or 16-bit formats to serve as inputs for the next layer.

Additionally, the eNPU3 hardware supports inline quantization for the output of an operator if required.

Convolution or pooling

eNPU3 supports inline quantization to convert the output to 8-bit or 16-bit before writing it to the memory. For convolution-based applications, it is recommended that the output of each layer be quantized to 8-bit in order to maintain the performance (GMAC/s) of the network. This also ensures that activation memory usage for the intermediate output is kept at a minimum.

Convolution

The following table lists the convolution requirements and the descriptions:

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyungchul.won@lge.com

Requirement	Description	Comments
Convolution input activations buffer format	Tensor format: • (Channel first, X, Y) • Activations [Yi] [Xi] [Ci]	Tensor data arranged with inner most dimension as channels, then X and Y.
Number of activation channels (Ci) without dilation	For 8-bit input: • 1 to 32: Ci <=32 • Multiple of 32: Ci >= 32	Channels are processed in groups of 32. Special handling for channels < 32.
	For 16-bit input: • 1 to 16: Ci <= 16 • Multiple of 16: Ci >= 16	Channels are processed in groups of 16. Special handling for channels < 16.
	Max Ci is 992 for all bit-widths	-
Number of channels (Ci) with dilation	For more information on dilation limitations, see Convolution dilation or padding .	-
Number of output channels (Co)	• Co: Any multiple of 2 < 32 • Co: Multiple of 32, when Co > 32	If using pooling after convolution, output channels are more restrictive. For more information on pooling limitations, see Max or average pool .
Stride	1 to 16 on X and Y	-
Input buffer alignment	Buffer start address should be aligned to 16 bytes. Additionally, each new row of tensor (Xi * Number of channels (Ci)) should be aligned with 16 byte.	Row alignment issue only occurs when Ci is not a multiple of 16. Zero padding will be added at eAI runtime if each input activation row is not aligned.
Output buffer alignment	Buffer start address should be aligned to 16 bytes. Additionally, each new row of output tensor ((Xo) * Number of channels (Co)) should be aligned with 16 byte.	Row alignment issue only occurs when Co is not a multiple of 16. Zero padding will be added at eAI runtime if each output activation row is not 16 byte aligned.
Weight buffer alignment	Buffer start address and each Kernel start address should be aligned to 16 bytes.	If Kernel is a multiple of 16 channels, each kernel is aligned to 16 byte by default.
Convolution padding	Only zero padding is supported.	Padding to achieve same input height and width for output.

Convolution weights

The following table lists the convolution weight requirements and the descriptions:

Requirement	Description	Comments
Number of Kernels (Ko)	• 1 - 32 in multiple of 2 (8-bit activation) • 1 - 16 in multiple of 2 (16-bit activation) • 32 - 992: Multiple 32 (8-bit activation) • 16 - 992: Multiple of 16 (16-bit activation)	Any other Ko values will default to either software implementation or offline model builder may add additional Kernels to meet requirements.
Number of weight channels (Kc)	Follow the same limitation as activation channels (Ci), as they need to match with the Convolution operation.	For 8-bit activations, a multiple of 32 channels is recommended for best hardware offload. Any non-multiple of 32 channels may result in software and scratch memory overhead due to runtime padding of activations before eNPU offload. Overhead will depend on input activation size.
Kernel size(K = #wt. Channels, Kc * Kx * Ky)	Each kernel is expected to be aligned with 16 bytes start address. Padding (zeros) will be added between each kernel by eAI runtime of the offline model builder if they are not aligned.	This situation would only arise when Kc is not multiple of 32 channels.
Max filter size (Kx and Ky)	• 1 <= Kx <= 127 • 1 <= Ky <= 127	-

Convolution examples for eNPU3 offload

The following table lists the convolution examples for eNPU3 offload:

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyungchul.won@lge.com

Convolution examples			eNPU optimized or memory overhead	Comments
Input dim Yi Xi Ci	Wt. Kernel Dim Ko Ky Kx Kc=Ci	Output Dim Yo Xo Co=Ko	-	-
240 240 1	32 3 3 1	240 240 32	Fully weight alignment padding	- Each input row (Xi * Ci) is 16 byte aligned. - Number of output channels are multiples of 32. - Each weight Kernel size (Ky * Kx * Kc) is 9 bytes. Memory padding will be added offline to make subsequent kernels start address 16 byte aligned.
120 120 16	32 3 3 16	120 120 32	Fully	- Each input row (Xi * Ci) is 16 byte aligned. - Number of output channels

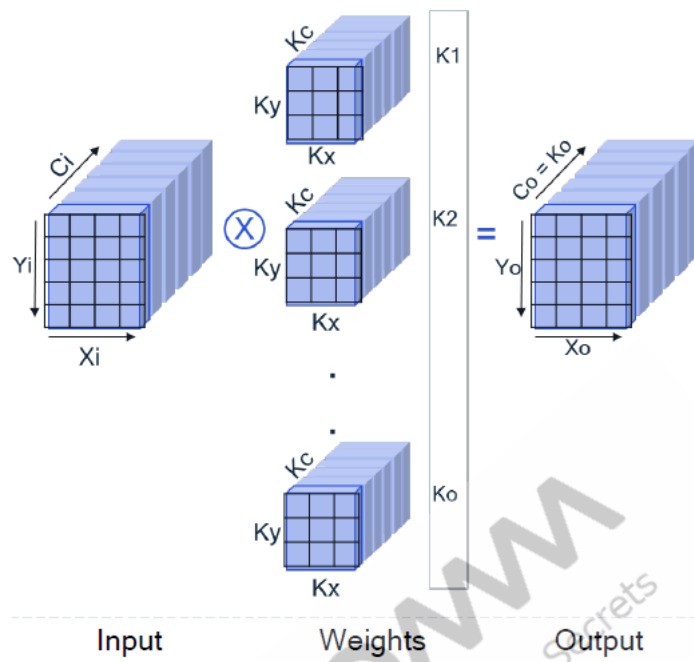


Figure3 Convolution examples

Note:

- For layers whose dimensions do not comply with eNPU hardware guidelines, the offline model builder applies padding to the weight tensors, which may lead to increased memory usage.
- Additionally, input and output tensors might require runtime padding.

Convolution dilation or padding

The eNPU3 natively supports only zero padding. When zero padding is used, it is important that the activations use symmetric quantization, as the zero value in the activation must align with the zero-padded value in hardware during convolution operations.

The following table lists the dilation requirements and their descriptions:

Dilation requirement	Description	Comments
Dilation width and height	1 to 4 on width (horizontal) and height (vertical). Should be same on both X and Y dims.	Default value is no dilation.
Kernel channels Kc	- Kc $\geq$ 32 (in multiple of 32) is preferred. - Kc < 32: Offline model builder padding overhead to weights tensor.	-

The following table lists the convolution padding requirements and their descriptions:

Convolution padding requirement	Description	Comments
Convolution padding type	Zero padding	-
Horizontal and vertical padding	0 to 31 on left or right and top or bottom.	- Left or right and top or bottom convolution padding can be independently disabled. - Left and right padding value when both enabled can only be different by 1. - Top and bottom padding value when both enabled can only be different by 1.

eNPU3 offload

Depth-wise convolution

The following table lists the depth-wise convolution requirements and their descriptions:

Depth-wise convolution requirement	Description	Comments
Number of activation channels (C_i)	• $32 \leq C_i \leq 992$ in multiple of 32 for 8-bit activation or kernel. • $16 \leq C_i \leq 992$ for 8-bit activation or kernel in multiple of 16.	-
Weight Kernel size ($K_x * K_y$)	If Kernel size is not multiple of 128-bits (16 byte), padding (zeros) needs to be added at the end of each kernel.	Hardware requirement. alignment

All other parameter limitations remain the same as in regular convolution.

The following table lists the examples for depth-wise convolution:

Depth-wise convolution examples			eNPU3 optimized or memory overhead	Comments
Input dim Yi Xi Ci	Wt. Kernel Dim Ko=Ci Ky Kx Kc=1	Output Dim Yo Xo Co=Ko	-	-
64 80 8	8 3 3 1	64 80 8	Not offloaded	Ci is not multiple of 32.
120 120 32	32 3 3 1	120 120 32	Fully offline weight alignment padding	Kx * Ky is 9 bytes. Additional 7 bytes padding will be added to the weights by offline model builder.
120 120 48	48 3 3 1	120 120 48	Partially	• Kx * Ky is 9 bytes. Additional 7 bytes padding will be added by model builder. • Number of channels not a multiple of 32. Additional zero channels in input and output will be added by model builder or runtime software.

Max or average pool

The following table lists the pooling requirements and their descriptions:

Pooling requirement	Description	Comments
Number of activation channels: Ci	• 8, 16, and any multiple of 32 • Maximum size: 992	Ci <= 32 has more restrictions in max or average pool compared to convolution operation.
Stride length	1 to 16	-
Max Kx, Ky size	1 to 127	-

The following table lists the max pool or avg pool convolution examples:

Max pool or avg pool convolution examples			eNPU3 optimized or memory overhead	Comments
Input dim Yi Xi Ci	Stride	Output Dim Yo Xo Co=Ko	-	-
120 160 24	2x2	60 80 24	No Offload	• Number of activation channels are not 8, 16, or multiple of 32. • maxpool may fall back to software implementation.
120 120 32	2x2	60 80 32	Fully	-
60 80 144	4x4	15 20 144	No Offload	• Number of activation channels are not 8, 16, or multiple of 32. • maxpool may fall back to software implementation.

Matrix based operators

Matrix based operators are utilized in fully connected layers, LSTM, GRU, attention mechanisms, and other similar architectures.

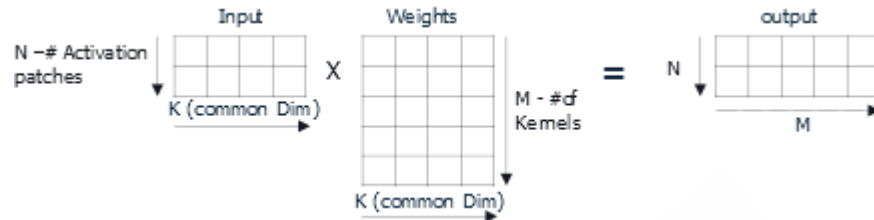


Figure4 Matrix based operators

The following table lists the different dimensions with the operations and the hardware supported values:

Dimension	Operations	Hardware supported values	Recommendations
N - Number of activation patches	GEMM	1 to 128	Higher values of N will result in multiple eNPU jobs and some processor overhead.
M - Number of Kernels	GEMM or GEMV	• $16 \leq M \leq 4080$ in multiple of 16 for 8-bit output. • $8 \leq M \leq 4080$ in multiple of 8 for 16-bit output.	Use hardware supported values. Additional processing on processor if M is not multiple of 16 or 8. Higher values of M than 4080 will result in multiple eNPU jobs.
K - Kernel size or common dimension	GEMV (N=1)	• Min value of K: 16 • $16 \leq K \leq 16368$ in multiple of 16 in 8x8b mode. • $16 \leq K \leq 8184$ in multiple of 8 in 16x16b mode. • $16 \leq K \leq 8184$ in multiple of 16 in 8x16b mode.	High values of K (common Dim) will result in multiple eNPU jobs and some processor computations to combine the results. If K is not multiple of 16, it will result in zero padding or additional processor computations.
	GEMM	• $16 \leq K \leq 4080$ in multiple of 16 in 8x8-bit and 8x16b modes. • $16 \leq K \leq 2040$ in multiple of 8 in 16x16b.	High values of K (common Dim) will result in multiple eNPU jobs and some processor computations to combine the results. If K is not multiple of 16, it will result in zero padding or additional processor computations.
Input or weight or output start address	GEMM or GEMV	All start addresses should be aligned to 16 byte boundary.	eAI modelbuilder will ensure this memory alignment.

Fully connected (GEMM or Matmul) examples



Figure5 Connected GEMM or Matmul examples

The following table lists the fully connected examples for 8-bit:

Qualcomm
 Confidential - May Contain Trade Secrets
 2026-05-12 06:35:39 GMT
 hyungchul.won@lge.com

Fully connected examples (8-bit examples)					eNPU offload or memory optimized ¹	Comments
Input Patches N	Common Dim K	Number of Kernels M	Output Dim	Output bit-width	-	-
1	64	32	1 X 32	8	Fully	-
1	60	32	1 X 32	8	Partially	Common Dim (K) is not multiple of 16. Some processing will be done on processor.
10	128	40	1 X 40	8	No Offload	Number of Kernels (M) is not multiple of 16. Processing will be done on processor.
10	128	40	3 X 40	16	Fully	Number of Kernels (M) is multiple of 8 for 16-bit output. Common Dim (K) is multiple of 16.

Element-wise ADD/SUB/MPY

Element-wise addition, subtraction, or multiplication of two vectors will have the same vector size for both inputs.

The following combinations are supported:

Operation	Supported (input1, input2) bit-width combinations
Multiply	(8, 8) (8, 16), (16, 8) (16, 16)
ADD or SUB	(8, 8), (16, 16)

The following table lists the attributes and requirements for element-wise addition, subtraction, or multiplication:

Attributes	Requirement	Comment
input_size (Ki) = output_size (Ko)	8-bit Input: Ki multiple of 16	Any non-multiple of Ki will result in additional processor computations.
	16-bit input: Ki multiple of 8	
	16 ≤ Ki ≤ 16384 for 8-bit 16 ≤ Ki ≤ 8192 for 16-bit	



Figure6 Element-wise ADD/SUB/MPY

For example:

¹ For layers whose dimensions do not follow eNPU3 hardware guidelines, additional DSP processing and scratch memory will be required to ensure proper alignment.

Attributes	Requirement	Comment
input_vector_size (Ki) = output_vector_size (Ko)	Ki to be multiple of 16 for 8-bit input vectors and multiple of 8 for 16-bit input vectors.	Any non-multiple of Ki will result in full or partial DSP fallback for this operation.

Element-wise ADD		eNPU3 optimized	Comments
Input1 and Input2 size (Ki)	Input bit width	-	-
82	8	No offload	Ki is not multiple of 16
96	16	Fully offloaded	Ki is multiple of 8

LSTM layer

Multiple operations in LSTM (GEMM, element-wise addition or multiplication with activation) are converted into distinct eNPU3 configurations. These configurations are then linked and offloaded to eNPU3 as a linked list of operations.

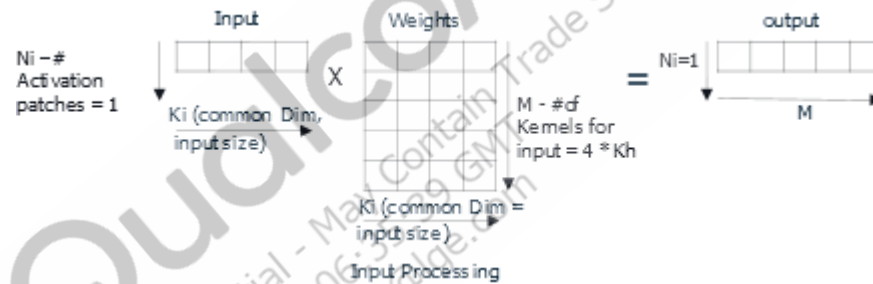


Figure7 LSTM layer input processing

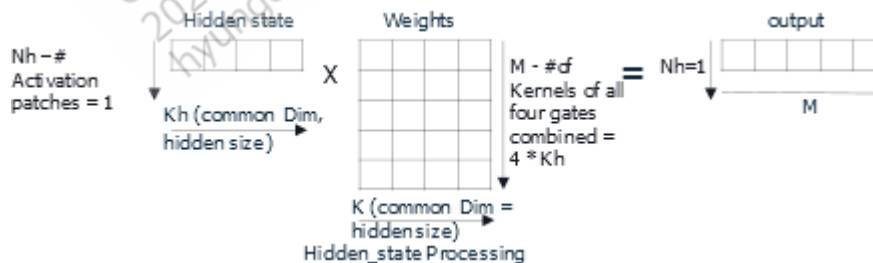


Figure8 LSTM layer hidden state processing

The following table lists the attributes of the LSTM layer and the requirements:

Attributes	Requirement	Comments
input_size (Ki)	Same as K (common dimensions) for fully connected GEMV operation.	Input tensor is used as input to GEMV operation.
hidden_size (Kh)	Same as K (common dimensions) for fully connected GEMV operation.	Hidden state tensor is used as input to GEMV operation.

The following table lists the LSTM examples for 8-bit weights:

LSTM examples (8-bit weights)		eNPU3 offload or memory optimized ²	Comments
hidden_size (Kh)	input_size (Ki)	-	-
128	320	Fully	Both hidden_size and input_size are multiples of 16.
96	320	Fully	Both hidden_size and input_size are multiples of 16.
88	96	Partially	hidden_size is not multiple of 16.
128	328	Partially	input_size is not multiple of 16.

The following table lists the LSTM examples for 16-bit weights:

LSTM examples (16-bit weights)		eNPU3 offload or memory optimized ³	Comments
hidden_size (Kh)	input_size (Ki)	-	-
128	320	Fully	Both hidden_size and input_size are multiples of 8.
96	320	Fully	Both hidden_size and input_size are multiples of 8.
88	96	Fully	Both hidden_size and input_size are multiples of 8.
128	324	Partially	input_size is not multiple of 8.

² For layers whose dimensions do not follow the eNPU3 hardware guidelines, additional DSP processing and scratch memory will be required for alignment.

³ For layers whose dimensions do not follow the eNPU3 hardware guidelines, additional DSP processing and scratch memory will be required for alignment.

GRU layer

Multiple operations in GRU such as GEMM, element-wise addition or multiplication with activation are converted into separate eNPU3 configurations. These configurations are linked together and offloaded to eNPU3 as a linked list of operations.

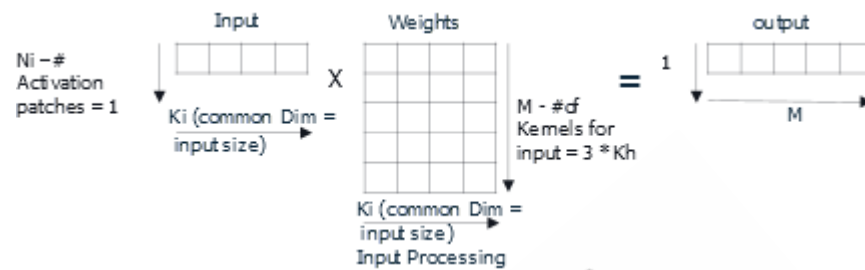


Figure9 GRU layer input processing

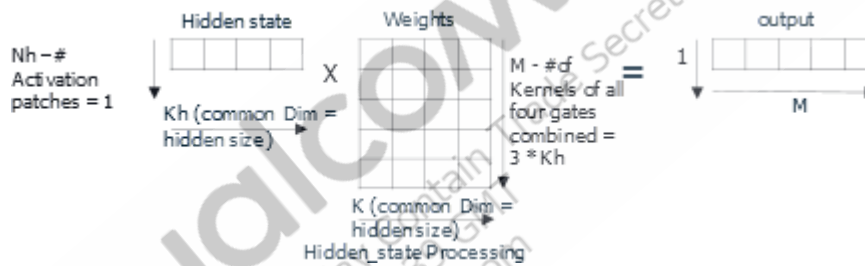


Figure10 GRU layer hidden state processing

The following table lists the attributes of the GRU layer and the requirements:

Attributes	Requirement	Comments
input_size (Ki)	Same as K (common dimensions) for fully connected GEMV operation.	Input tensor is used as input to GEMV operation.
hidden_size (Kh)	Same as K (common dimensions) for fully connected GEMV operation.	Hidden state tensor is used as input to GEMV operation.

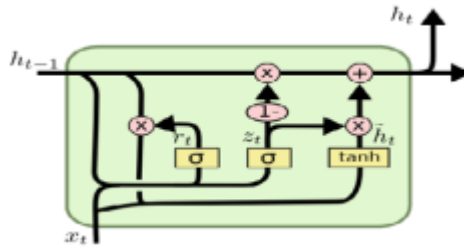


Figure11 GRU layer equation logic

The following table lists the GRU examples for 8-bit weights:

GRU examples (8-bit weights)		Optimized ⁴	Comments
hidden_size (Kh)	input_size (Ki)	-	-
128	320	Fully	Both hidden_size and input_size are multiples of 16.
96	320	Fully	Both hidden_size and input_size are multiples of 16.
88	96	Partially	hidden_size is not multiple of 16.
128	328	Partially	input_size is not multiple of 16.

The following table lists the GRU examples for 16-bit weights:

GRU examples (16-bit weights)		Optimized ⁵	Comments
hidden_size (Kh)	input_size (Ki)	-	-
128	320	Fully	Both hidden_size and input_size are multiples of 8.
96	320	Fully	Both hidden_size and input_size are multiples of 8.
88	96	Fully	Both hidden_size and input_size are multiples of 8.
128	324	Partially	input_size is not multiple of 8.

⁴ For layers whose dimensions do not follow the eNPU3 hardware guidelines, additional DSP processing and scratch memory will be required for alignment.

⁵ For layers whose dimensions do not follow the eNPU3 hardware guidelines, additional DSP processing and scratch memory will be required for alignment.

4 Offloading operations using embedded Neural Processing Unit

The S7/S7+ Gen 1 sound platform supports a dedicated ML accelerator called an embedded Neural Processing Unit (eNPU) version 3, which allows the eAI runtime to offload some of the neural network operations to achieve low power and better performance.

This hardware accelerator is designed to assist machine learning inference framework running on the processor. However, it has specific limitations regarding layer parameters and tensor dimensions across various operators.

When a layer does not meet these limitations, the following mechanisms should be applied:

- Additional padding may be added during the eAI model conversion process.

or

- Pre-processing or post-processing will be done in the Core/HAL running on the processor.

or

- The entire layer may be executed on the processor within the eAI framework.

These can lead to suboptimal execution of the layer on the eNPU.

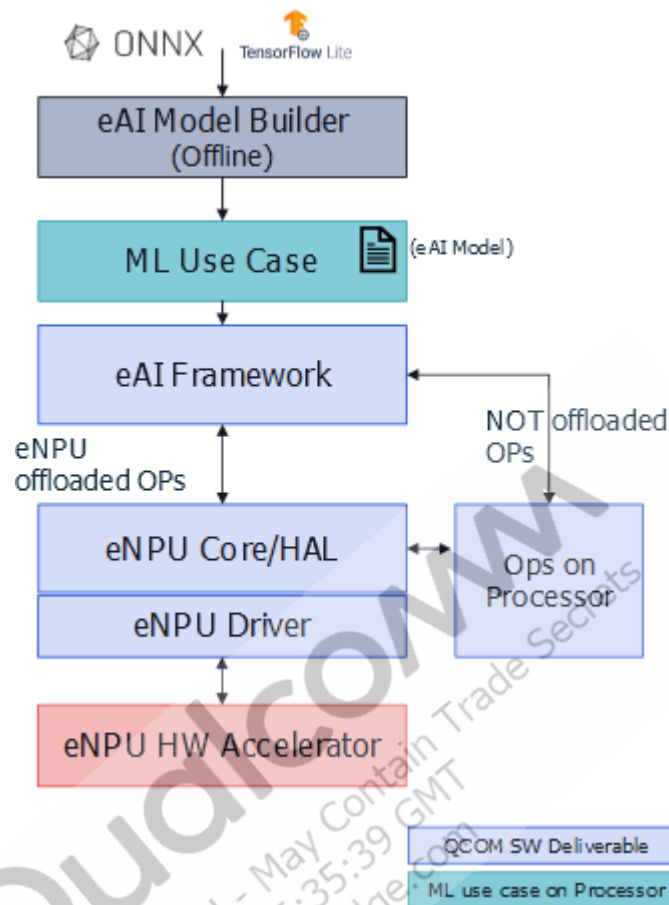


Figure1 eNPU workflow

The eNPU is paired with the HiFi4 DSP in S7/S7+ Gen 1 sound platform and controlled from the eAI runtime. To meet reduced power in DNN processing with a more minor memory requirement, eNPU requires 8-bit or 16-bit quantization for weights and activation inputs. Building the model with eNPU support also does weight compression and optimization to save on bandwidth and memory footprint.

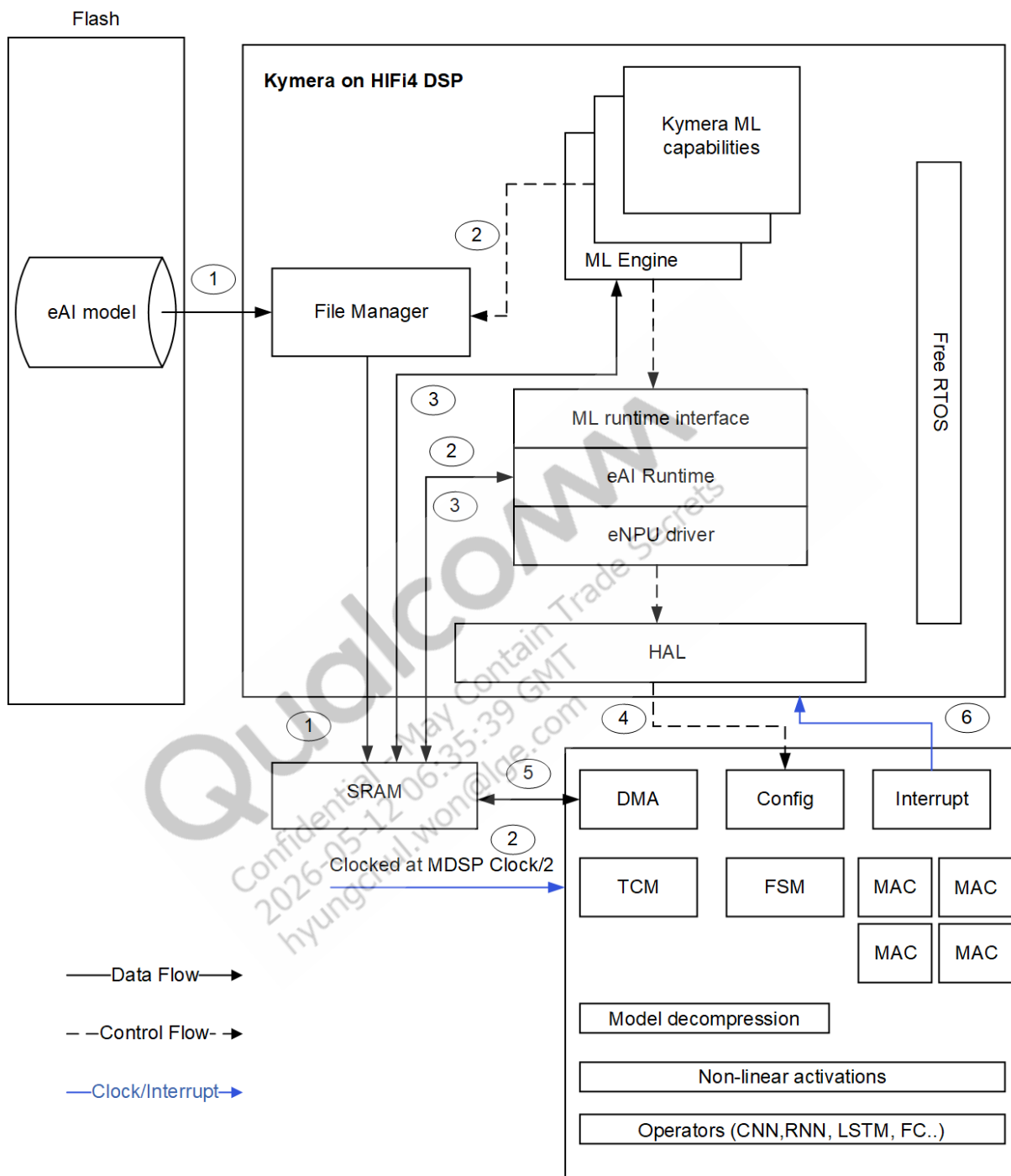


Figure2 Machine learning offloading

For the supported list of offloaded operators and quantization requirements, see the online eAI toolkit documentation.

4.1 eNPU data flow

The programming model and data flow with eNPU is as follows:

1. Load the model to the Audio framework. For the API reference, see the *Kymera API documentation*. The following steps are typically required to load the model to the Audio framework:
 - a. Add the eAI model created using the eAI model builder with eNPU optimization to the file system on Flash.
 - b. Enable the Audio subsystem from the application using `OperatorFrameworkEnable` API.
 - c. Load the model to the Audio subsystem using `OperatorDataLoadEx` API.
2. Initialize eAI and load the model through the Kymera operator associated with the model. For ML Engine capability operator messages, see the *Kymera Capability User Guide*. The following steps are required to load the model to the operator:
 - a. Create the ML Kymera operator associates with the model and set up the audio graph.
 - b. Load the model to the ML operator using `LOAD` operator message. The Operator message must specify file handle id from the `OperatorDataLoadEx` API and if the model has offload enabled.
 - c. The Operator initializes eAI and during initialization, eAI runtime parses the model graph and creates hardware and software OP list, and configures OPs for eNPU and DSP.
 - d. If there are OPs to be offloaded, eAI runtimes start the eNPU. eNPU runs at half the speed DSP runs (for example, at a maximum, DSP runs at 500 MHz and eNPU at 250 MHz).
3. The ML capability does the model inference per frame while processing the audio data.
 - a. When the use case is running, the audio graph process data. The ML capability preprocesses the audio frames and prepares activation input tensors in the SRAM.
 - b. The `ml_execute` API invokes the `eai_execute` API to start. eAI executes the non-offloaded operators graph.
4. eAI runtime invokes eNPU and offloads:
 - a. While eAI executing the graph, it prepares workload for eNPU for the supported offload operators.
 - b. eAI invokes eNPU processing through the driver. eAI can offload up to four jobs simultaneously.
5. eNPU processes each OP by reading input and weights from shared memory and generating output into shared memory.
6. An interrupt is generated after each Op or group of Ops. After receiving the interrupt, eAI

runtime accesses the output from SRAM and processes it along with the rest of the eAI graph. This repeats until processing the entire eAI graph before it provides a status to the ML Kymera operator. It can access the output from shared RAM buffer and post-process it before feeding it to the next operator in the Kymera graph.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyungchul.won@lge.com

5 Machine Learning runtime interface and APIs

The ML runtime interface is an adaption layer between the eAI runtime interface and Kymera ML capability. It associates the capability with the model and simplifies the sequence by adapting to a typically used sequence.

Note: For the eAI API definitions, see the online documentation with the eAI tools.

Machine Learning programming APIs

API	Description	Internal eAI APIs
ml_load()	Loads the model and initializes the instance with default settings.	eai_init(), eai_get_property(), eai_set_property(), eai_pre_apply()
ml_unload	Unloads the model and frees the context.	eai_deinit()
ml_set_config()	Configures the model properties.	eai_set_property(), eai_apply()
ml_get_config()	Gets the configured model properties.	eai_get_property()
ml_execute()	Executes the model, also known as model inference.	eai_execute()
ml_get_eai_init_f()	Optional utility API. Gets the eAI utility interface. This is for the clients to override the properties handled by the ML runtime and to interface directly with eAI.	-

The following table describes the process to load the model.

bool ml_load(unsigned usecase_id, ML_MODEL_BUFFER * model, ML_INIT_FLAGS *init_flags, ML_CONTEXT **context)		
Description	Loads a single model to RAM and initiates eAI context and eNPU if the model supports offloading.	
Params	usecase_id	Usecase identifier associated with the model.
	model	Pointer to the model buffer.
	init_flags	Pointer to initialisation settings. If this is NULL, default values are applied. One of the flags is use_mla; it must be set if the model was created with eNPU optimisation.
bool ml_load(unsigned usecase_id, ML_MODEL_BUFFER * model, ML_INIT_FLAGS *init_flags, ML_CONTEXT **context)		
	context	The context returned after a successful initialisation. This context handle must be used for any further operations with this model.
Returns	TRUE if the model was successfully loaded and eAI context was initialised.	

The following table lists the configuring model properties.

<pre>bool ml_set_config(ML_ CONTEXT *ml_context, PropertyID property_id, ML_ PROPERTY *property)</pre>		
Description	A common interface to set the model properties. Without calling this API, the default configurations are applied.	
Params	ml_context	The context returned in ml_load().
	property_id	An identifier for the property to be set. The supported properties are defined in ml_struct.h
	property	The property data to be set.
Returns	TRUE on successfully setting the provided model property.	
<pre>bool ml_get_config(ML_ CONTEXT *ml_context, PropertyID property_id, ML_PROPERTY *property)</pre>		
Description	A common interface to get the applied model properties.	
Params	ml_context	The context returned in ml_load().
	property_id	An identifier for the property to get.
	property	The property data to be populated.
Returns	TRUE on successfully getting the provided model property.	

The following table describes the process to execute the model.

bool ml_execute(ML_CONTEXT *ml_context, ML_BATCH_INFO *batch_info, bool *more)		
Description	Executes the model.	
Params	ml_context	The context returned in ml_load()
	batch_info	If set to NULL, it assumes a single batch using scratch buffers.
	more	Populate to TRUE if more data is expected.
Returns	TRUE on successfully executing the model.	

The following table describes the process to unload the model.

bool ml_unload(ML_CONTEXT **ml_context)		
Description	Unloads the model and free the context.	
Params	ml_context	The context returned in ml_load()
Returns	TRUE on successfully unloading the model and freeing the context.	

6 Kymera Machine Learning capability

The Kymera ML Capability is a Kymera capability with additional code to access the eAI model, prepare the input activation tensors (pre-processing), and process the output tensors (post-processing) and model inference by calling the ML or eAI runtime interface.

The ML capability bundle includes two example capabilities:

- ML Base capability
- ML Engine capability

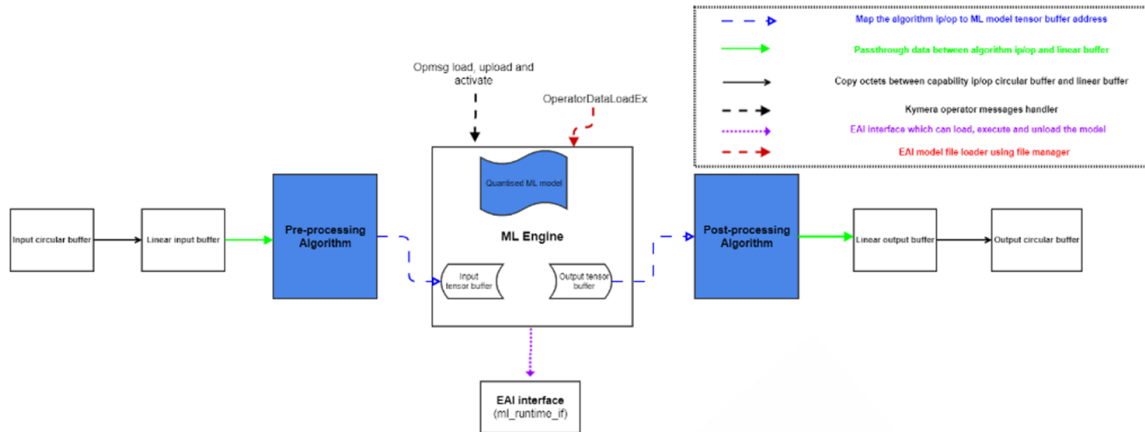
6.1 Machine Learning base capability

Using an ML Base template capability, the ML framework enables developers to deploy their ML models on supported target audio platforms.

This template capability includes placeholders for:

- ML pre-processing: Creates the required input tensors for a specific ML model by extracting the features from the input audio data to the capability using provided ML pre-processing algorithms.
- ML engine runs the Inference ML model on the input tensors to generate output tensors and is also a library that includes utility functions to prepare data for eAI/ML interface in the correct sequence.
- ML post-processing: Converts the ML model output tensors into application events or audio data that can then be connected to a subsequent Qualcomm Kymera Operator or an Audio Endpoint, depending on the graph structure.

The following figure shows the ML base capability, which can be used as a reference or skeleton to build custom ML capabilities.



The following are the benefits of using the ML base capability:

- Provides a working example of creating an ML capability for Kymera. This includes everything necessary for a standard Kymera capability.
- Shows how to use the public APIs the Kymera ML framework exposes.
- Has all the necessary operator messages for configuring the kymera ML engine.

The ML base capability covers two processing algorithms for demonstration purpose:

- Data passthrough: Passthroughs data from input to output.
- FFT: Applies nature DSP FFT libraries to implement FFT in the preprocessing algorithm and IFFT in postprocessing algorithm. Normalization of output audio occurs after IFFT.

6.2 Machine Learning engine capability

The Machine Learning Engine capability is a reference capability that implements easy-to-use utility functions.

It allows:

- Loading and activating an eAI model.
- Connecting input and output streams that expect activation input tensors at input.
- Generating output tensors in its raw format.

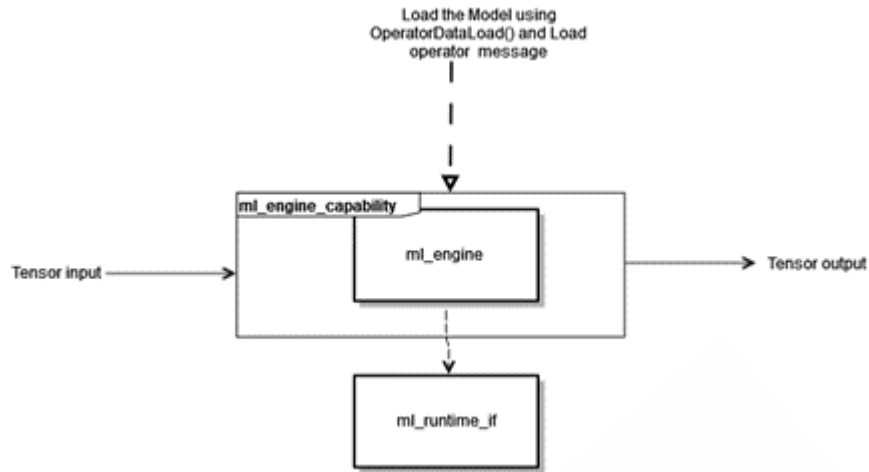


Figure1 Machine Learning engine capability

Qualcomm
 Confidential - May Contain Trade Secrets
 2026-05-12 06:35:39 GMT
 hyungchul.won@lge.com

Operator messages

The Kymera ML Engine capability implements operator messages that any Kymera ML capability implementation can use.

For examples of handling these messages, see [Machine Learning base capability](#) and [Machine Learning engine capability](#) implementations.

The following table lists the Kymera ML Engine capability operator messages.

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyungchul.won@lge.com

Message_id	Function	Arguments	Usage
0x0002	OPMSG_ML_ ENGINE_ID_ SET_INPUT_TEN SOR_SEQUENCE	• usecase_id: Usecase_ID associated with the loaded file • num_ip_tensor: Number of input tensors • tensor_sequence: Sequence of the input tensor in the input buffer - list of tensor_ids • input buffer: List of tensor_ids	This message informs MLE about the sequence of the input tensors in its input buffer. The ML Engine uses this to know the order and the boundaries of the tensors in a frame.
0x0003	OPMSG_ML_ ENGINE_ID_ SET_OUTPUT_TEN SOR_SEQUENCE	• usecase_id : Usecase_ID associated with the loaded file • num_op_tensor: Number of output tensors • tensor_sequence: Sequence of the output tensors expected in the output buffer - list of tensor_ids	This message informs MLE about the sequence of the output tensors expected in its output buffer. The ML Engine uses this to reorder output tensors and its boundaries.
0x0014	OPMSG_ML_ ENGINE_ID_ LOAD_MODEL	• usecase_id: Usecase_ID associated with the loaded file • batch_reset_count: Batch execution count after which persistent tensors need to be reset • file handle:	Loads the model file

To select the correct algorithm in ML base, it provides the additional operator message `OPMSG_ML_BASE_ID_SET_ALGORITHM_TYPE` to select which algorithm to run (0 = data passthrough and 1 = FFT). Data passthrough is run by default.

Self-test mode

The ML Engine capability exposes a self-test mode for model validation on hardware without connecting the capability to an audio graph using the raw data for the input tensors, the golden output tensors. It is normally used for model benchmarking, power measurement and performance analysis. The ML engine self-test mode allows running the ML engine capability in a standalone mode without connecting input or output buffers.

Note the following about the self-test mode:

- The `LOAD_MODEL` operator message allows the loading of the model. The user can run as many models as possible (and use cases).
- The model does not need to be activated for running the self-test mode because the self-test always activates the model associated with the test before running it and deactivates it at the end of the test.
- The `SELF_TEST_MODE` message must be provided with an activation input file index and golden output file index from `OperatorDataLoadEx`.
- Starting the operator infers the model and runs until emptying the activation input file.
- The output buffer is compared against the golden output file, and if it matches, the overall performance, including test status, total cycles for all inference and offloaded layers are returned in the log.

Performance analysis

Except for the returned overall performance, more detailed analysis including layer-wise cycles, model outputs and so on is accessible in KDAT ml analysis. Furthermore, eNPU performance analysis of each offloaded job is displayed in the KDAT debuglog analysis. For more information, see [Kymera Machine Learning example application](#).

Power measurement

To facilitate the power measurement stage, `num_inferences` can be set in `OPMSG_ML_ENGINE_ID_START_SELF_TEST` to run the same inputs multiple times. For example, if the model takes 100 us to inference, the `num_inferences` can be set to 10000. The benefit of this approach is that no extra memory is required for additional inputs or goldens and power can be averaged over 1 second to get more reliable results.

6.3 Kymera Simulation Environment validation

This section describes the various usecases when two ML reference capabilities are run on the Kymera Simulation Environment (KSL) shell. To install and develop KSL, see the *Qualcomm Kymera Simulation Environment (KSL) User Guide*.

- Case 1: ML Engine self-test mode with internal sample offloaded model.
- Case 2: pcm (audio source endpoint) → ML base (data passthrough) → pcm (audio sink endpoint) with non-offloaded gain model (gain ratio = 0.5).
- Case 3: pcm (audio source endpoint) → ML base (FFT) → pcm (audio sink endpoint) with offloaded gain model (gain ratio = 2).
- Case 4: Use `OperatorDataLoadEx` utility function to load custom models and input tensors/golden tensors from resource files to local audio memory and play with MLE self-test mode.

For demonstration in Case 1 and Case 2, two models are stored internally in the capabilities, where providing file handle 0 in load model `opmsg` means that sample model and 1 means that gain model.

For FFT demonstration in case 3, the `fc2_hw.eai` model (located in `<ADK_ROOT>\example_apps\examples\src\ml\`) is used to amplify the output waveform by 2.

For Case 4, the `OperatorDataLoadEx` utility function can be called to load any custom files outside to local memory, which gives a special file handle.

Outputs are:

- Performance analysis of MLE self-test mode in `\workspace\log\shell.log`.
- Output waveform of MLB pcm loopback in `\workspace\tmp`.
- Layer-wise cycles can be accessible in KDAT ml analysis.

To get cycle-accurate performance, turbo mode is located at `workspace\config\ksl_shell.qccx22x_qccx32x.hifisim.cfg` should be turned to false (true by default).

Note:

- eNPU in simulation is not cycle-accurate.
- Loading downloadable capability or model files may take some time to respond so it is better to call separately or try several times.

MLE self-test mode with sample model

Run the following commands in KSL shell environment:

```
import time

# Duration for operator running time
TEST_STEP_DURATION = 5

# OPMSG IDs
OPMSG_ML_ENGINE_ID_LOAD_MODEL = 0x0014
OPMSG_ML_ENGINE_ID_START_SELF_TEST = 0x001B

# Download ML capability from firmware directory (download ml
includes both ML engine and ML base)
hydra_cap_download.download('firmware/download_ml.dkcs')

# Create ML Engine capability (operator ID = 0X40B1)
# it should return 0x4040 marked that this is a first operator
mle = kymera.opmgr_create_operator(0x40B1)

# Load the model (arguments in sequence are usecase id = 1, batch_
count = 0 (means 1 batch per inference), file_handle (0 means sample
model), use_offloading = 1 (uses eNPU offloading))
kymera.opmgr_operator_message(mle, [OPMSG_ML_ENGINE_ID_LOAD_MODEL, 1,
0, 0, 1])
print("Load the sample model in mle")

# Start the mle operator
kymera.opmgr_start_operators([mle])
print("start the operator!")

# Start the self-test mode (usecase id = 1, num_inferences = 1,
input_handle = 0 (load internal input tensors), golden handle = 0
(load internal golden tensors))
kymera.opmgr_operator_message(mle, [OPMSG_ML_ENGINE_ID_START_SELF_
TEST, 1, 1, 0, 0])

print("start the self-test sample model")
```

```

uut.timer_wait_relative(TEST_STEP_DURATION)

kymera.opmgr_stop_operators([mle])
print("stop mle operator ")

kymera.opmgr_destroy_operators([mle])
print("destroy mle operator ")

print("Test finished!!")

```

ML Base pcm loopback with non-offloaded gain model

Note: Gain ratio is 0.5.

1. Create a mlb.cfg JSON file, which should be similar to the pt.cfg that is provided by KSL but with ml base configuration. Important changes are in the operator section. Load model covers 4 arguments, including:

- usecase id 1
- batch reset count 0
- file handle 1 (gain model)
- 0 (non-empu offloaded)

```

{
  "stream": [
    {
      "interface": "pcm",
      "type": "source",
      "kwargs": {
        "backing": "file",
        "filename": "resource/153_Prompts_176.4_kHz_Music_
Detected_8k.raw",
        "sample_rate": 8000,
        "sample_width": 16
      }
    },
    {
      "interface": "pcm",
      "type": "sink",
      "kwargs": {
        "backing": "file",
        "filename": "tmp/output_0.wav",

```

```

        "sample_rate": 8000,
        "sample_width": 16
    }
}
],
"endpoint": [
{
    "interface": "pcm",
    "type": "source",
    "kwargs": {
        "instance": 0,
        "channel": 0,
        "pcm_sync_rate": 8000,
        "pcm_sample_rising_edge": 0,
        "pcm_master_mode": 1,
        "pcm_slot_count": 4,
        "pcm_sample_format": 1,
        "pcm_master_clock_rate": 512000
    }
},
{
    "interface": "pcm",
    "type": "sink",
    "kwargs": {
        "instance": 0,
        "channel": 0,
        "pcm_sync_rate": 8000,
        "pcm_sample_rising_edge": 0,
        "pcm_master_mode": 1,
        "pcm_slot_count": 4,
        "pcm_sample_format": 1,
        "pcm_master_clock_rate": 512000
    }
}
],
"operator": [
{
    "interface": "DOWNLOAD_ML_BASE",
    "args": [
        [
            "LOAD_MODEL",
            [
                1,
                0,

```

```

        1,
        0
    ],
    [
        "ACTIVATE_MODEL",
        [
            1
        ]
    ]
]
}
],
"graph": [
    [
        "stream0",
        "endpoint0"
    ],
    [
        "endpoint0",
        "operator0i0"
    ],
    [
        "operator0o0",
        "endpoint1"
    ],
    [
        "endpoint1",
        "stream1"
    ]
]
]

```

```

}

```

2. Open the ksl shell environment and perform the following steps:
 - a. Download the ML capability, which includes the ML base.
 - b. Load mlcfg.json and play the graph.

```
ksl > hydra_cap_download.download('firmware/download_ml.dkcs')
< 0x0001

ksl > from ksl.framework.library.file_util import load
.... data = load('config/mlb.cfg.json')
.... graph.play(data)
```

Figure2 Playing the graph

The output waveform should look like the input audio waveform shown in [Playing the graph](#), and degraded amplitude of the output waveform, as shown in the following figure.

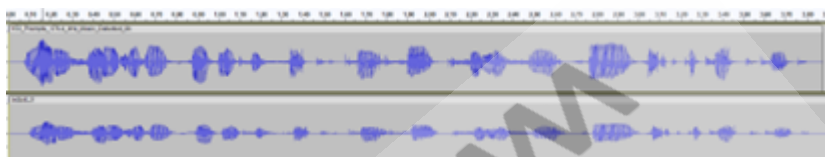


Figure3 Output waveform example

ML Base pcm loopback (FFT algorithm) with offloaded gain model (amplify by 2)

Instead of using internal sample model, it needs to load the model and get handle id (0x6c01), which is used by LOAD_MODEL opmsg of test configure file as shown in [ML Base pcm loopback \(FFT algorithm\) with offloaded gain model \(amplify by 2\)](#).

```
ksl > hydra_cap_download.download('firmware/download_ml.dkcs')
< 0x0001

ksl > def OperatorDataLoadEx(filename, file_type, storage, auto_remove):
.... hydra_file_manager.download(filename, file_type=file_type, file_location=storage, auto_remove=auto_remove)

ksl > handle = OperatorDataLoadEx('resource/fft_hu.eai', 0, 1, 1); handle
< 0x6c01

ksl > from ksl.framework.library.file_util import load
.... data = load('config/mlb_fft.cfg.json')
.... graph.play(data)
```

Figure4 ML Base pcm loopback (FFT algorithm) with offloaded gain model (amplify by 2)

Like normal loopback, mlb_fft.cfg.json file is created with extra opmsg to change data passthrough algorithm to FFT.

```
{
  "stream": [
    {
      "interface": "pcm",
      "type": "source",
      "kwargs": {
        "backing": "file",
```

```
        "filename": "resource/sine_716_10s_1c_8000_16b.wav",
        "sample_rate": 8000,
        "sample_width": 16
    }
},
{
    "interface": "pcm",
    "type": "sink",
    "kwargs": {
        "backing": "file",
        "filename": "tmp/output_0.wav",
        "sample_rate": 8000,
        "sample_width": 16
    }
}
],
"endpoint": [
    {
        "interface": "pcm",
        "type": "source",
        "kwargs": {
            "instance": 0,
            "channel": 0,
            "pcm_sync_rate": 8000,
            "pcm_sample_rising_edge": 0,
            "pcm_master_mode": 1,
            "pcm_slot_count": 4,
            "pcm_sample_format": 1,
            "pcm_master_clock_rate": 512000
        }
    },
    {
        "interface": "pcm",
        "type": "sink",
        "kwargs": {
            "instance": 0,
            "channel": 0,
            "pcm_sync_rate": 8000,
            "pcm_sample_rising_edge": 0,
            "pcm_master_mode": 1,
            "pcm_slot_count": 4,
            "pcm_sample_format": 1,
            "pcm_master_clock_rate": 512000
        }
    }
]
```



```
    }  
  ],  
  "operator": [  
    {  
      "interface": "DOWNLOAD_ML_BASE",  
      "args": [  
        [  
          "LOAD_MODEL",  
          [  
            1,  
            0,  
            27649,  
            1  
          ]  
        ],  
        [  
          "SET_ALGORITHM_TYPE",  
          [  
            1,  
            256,  
            256  
          ]  
        ],  
        [  
          "ACTIVATE_MODEL",  
          [  
            1  
          ]  
        ]  
      ]  
    }  
  ],  
  "graph": [  
    [  
      "stream0",  
      "endpoint0"  
    ],  
    [  
      "endpoint0",  
      "operator0i0"  
    ],  
    [  
      "operator0o0",  
      "endpoint1"
```

```

    ],
    [
        "endpoint1",
        "stream1"
    ]
]

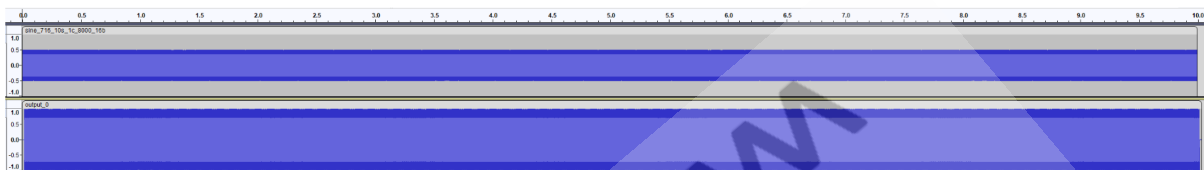
```

```

}

```

Output waveform should be double than input.



Custom model loading with ML Engine self-test mode

Run the following commands in KSL shell environment:

```

import time

OPMSG_ML_ENGINE_ID_LOAD_MODEL = 0x0014
OPMSG_ML_ENGINE_ID_START_SELF_TEST = 0x001B

TEST_STEP_DURATION = 5

# Download ML capability from firmware directory (download ml
includes both ML engine and ML base)
hydra_cap_download.download('firmware/download_ml.dkcs')

# Create ML Engine capability (operator ID = 0X40B1)
# it should return 0x4040 marked that this is a first operator
mle = kymera.opmgr_create_operator(0x40B1)

from enum import IntEnum

class FileType(IntEnum):
    bin = 1
    eAI = 2
    elf = 3

class FileLoc(IntEnum):
    sram = 1

```

```

class AutoRemove(IntEnum):
    no = 0
    yes = 1

# Utility function to load model/tensors from resource file to local
audio memory
# Filetype 1 means raw/binary file and 2 means eai file format
# Storage here 1 means store file in local memory
# auto_remove 1 means the file can be auto-removed later after
finishing all operations
def OperatorDataLoadEx(filename, file_type, storage, auto_remove):
    return hydra_file_manager.download(filename, file_type=file_type,
file_location=storage, auto_remove=auto_remove)

# Here, we use anomaly detection tinyML model as an example
ad_enpu_handle = OperatorDataLoadEx("resource/ad_hw.eai", FileType.
eAI, FileLoc.sram, AutoRemove.yes)
print("load the ad eNPU model from file manager: ", hex(ad_enpu_
handle))
ad_input = OperatorDataLoadEx("resource/ad_input.raw", FileType.bin,
FileLoc.sram, AutoRemove.yes)
ad_golden = OperatorDataLoadEx("resource/ad_golden.raw", FileType.
bin, FileLoc.sram, AutoRemove.yes)
print("load the ad input handle, golden handle", hex(ad_input),
hex(ad_golden))

# configuration parameters
usecase_id = 2
batch_reset_count = 0
use_mla = 1
num_of_inferences = 1

# Load the ad model by provided file handle ID
kymera.opmgr_operator_message(mle, [OPMSG_ML_ENGINE_ID_LOAD_MODEL,
usecase_id, batch_reset_count, ad_enpu_handle, use_mla])
print("Load the ad eNPU model in mle")
kymera.opmgr_start_operators([mle])
print("start the operator!")
kymera.opmgr_operator_message(mle, [OPMSG_ML_ENGINE_ID_START_SELF_
TEST, usecase_id, num_of_inferences, ad_input, ad_golden])
print("start the self-test ad model")
uut.timer_wait_relative(TEST_STEP_DURATION)

```

```
kymera.opmgr_stop_operators([mle])  
print("stop mle operator ")  
  
kymera.opmgr_destroy_operators([mle])  
print("destroy mle operator ")  
  
print("Test finished!!")
```

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyungchul.won@lge.com

7 Kymera Machine Learning example application

The ML example application can be configured in three modes:

- ML loopback mode
- ML loopback with FFT mode
- ML Engine self-test mode

The ML loopback application can be used as a reference for developing audio applications using Kymera ML capabilities. The ML capability bundle includes two capabilities:

- ml_engine
- ml_base

The ML engine self-test application uses the ml_engine capability to validate a new model on hardware without implementing an audio graph. It can also be used for model benchmarking and power measurement in hardware. The application source can be found in the ADK at

`<ADK_ROOT>\example_apps\examples\src\ml\.`

For information about application development workflow using the ADK, see the *ADK Software Getting Started Guide*.

This section describes the Machine Learning application setup configuration, execution and usage of KDAT to debug and generate ML analysis for model performance.

7.1 Setting up Machine Learning example application

Since the ML application is part of the example application, it needs to be turned on by running the python commands to update examples.h and examples_private.h files for ML application configurations. For more information, see `<ADK_ROOT>\example_apps\examples\examples.md`.

Note: Do not enable audioloopback example at the same time as the ML example.

`examples_update.py` should only include `ML.py` `examples_update.py` `ml`

#. Turn on a particular ML application mode. The correct macro should be defined in `example_apps\examples\src\ml\ml.h`. By default, the ML loopback is selected.

- `#define ML_LOOPBACK`: Runs normal ml loopback (default setting).
 - `#define ML_LOOPBACK` and `#define ML_FFT_ALGORITHM`: Runs ML loopback with FFT implementation.
 - `#define ML_ENGINE_SELF_TEST`: Runs self-test mode. This mode allows two submodes:
 - `#define INFERENCE_PROFILING` mode: Does not allow the DSP to sleep during offload to record the overall cycles taken by both DSP and eNPU.
 - `#define POWER_PROFILING` mode: Allows DSP to enter into low power mode on S7/S7+ Gen 1 sound platform, which is useful for power profiling at Max DSP clock of 500 Mhz. The `NUM_INFERENCE` can be changed to run the test for a longer duration.

The following sample models and supporting files are provided along with the ML example application:

- `fc2_hw.eai`: Sample model for ml loopback with FFT. This is a fully connected layer with a diagonal weight matrix of multiplier two. The model gets offloaded to the eNPU3.
- `gain_sw.eai`: Sample model for ml loopback. It is a gain model with -6 db gain. This is a non-offloaded model.
- `vww_hw.eai`, `vww_input.raw` and `vww_golden.raw`: 8-bit visual waking word eNPU3 offloaded model converted from MLPerf tiny model with sample input and golden output, all used for self-test mode.

1. Flash the files needed by the application. Append the entry in `FS_FILES_FOR_PROJ` in

`\example_apps\examples\Sconscrip.py`. For example, to run ML engine self-test mode, the model, input and golden output files must be included.

```
p_env.Append(FS_FILES_FOR_PROJ=[os.path.join(p_
env['ROOT'], 'audio/qcc722x/ audioss/kymera/prebuilt_dkcs/libretto2_
release/download_passthrough.dkcs'), os.path.join(p_
env['ROOT'], 'audio/qcc722x/ audioss/kymera/prebuilt_dkcs/libretto2_
release/download_ml.dkcs'), os.path.join(p_env['ROOT'], 'example_apps/
examples/src/ml/vww_hw.eai'), os.path.join(p_env['ROOT'], 'example_
apps/ examples/src/ml/vww_input.raw'), os.path.join(p_
env['ROOT'], 'example_apps/ examples/src/ml/vww_golden.raw')])
```

1. Build the example application in the ADK path. Application image is located in `example_apps`

`\examples\workspace\<chip variant>\output` and filesystem is located in

\example_apps\examples\workspace\<chip_variant>\filesystems. The example below shows building the application for the QCC7226-AA_DEV-BRD-AA variant.

```
vbuild QCC7226-AA_DEV-BRD-AA_EXAMPLES
```

1. Follow the ADK guidelines to flash this application image and other images, such as audio or Bluetooth, into the hardware.

NOTE For the ML loopback application, the connection with ANC Plug-In V3 Module and sound card are required for ADC/DAC mono setup.

7.2 Machine Learning example application

The application source code should be available under `example_apps\examples\src\ml\ml.c`. In all applications, only one model is used so the use case id used is always 1. Depending on the free heap size, the multiple models/use cases can be loaded but the `ml_engine` and `ml_base` capability provided only allows one use case to be activated at a time. All the apps configuration is set by `ml_configure` (`example_apps\examples\src\ml\ml.h`) structure below.

```
/* Configuration of the machine learning examples */
static const ml_config_t ml_config = {
    /* ML usecase id starting from 1 */
    .use_case_id = 1,
#ifdef ML_ENGINE_SELF_TEST
    /* Default capability ID of the ML engine capability */
    .capability_id = 0x40B1,
    /* Offloaded visual waking words TinyML model */
    .model_file = "vww_hw.eai",
    /* Model access methods (use enpu to offload or not) */
    .access_method = MODEL_ENPU_OFFLOADED,
    /* Run the input tensor once */
    .num_inferences = NUM_INFERENCES,
    .input_file = "vww_input.raw",
    .golden_file = "vww_golden.raw",
#endif //ML_ENGINE_SELF_TEST

#ifdef ML_LOOPBACK
    /* Default capability ID of the ML base capability */
    .capability_id = 0x40AF,
#ifdef ML_FFT_ALGORITHM
    /* Offloaded fully-connected layer with multiplier 2 on weights,
    running on 16-bit
    * frequency components. */
    .model_file = "fc2_hw.eai",
```

```

    /* Algorithm type 1 -> FFT, 256 octets are needed by processing
    block
    size to process
    * the 128 fft length on 16-bit model io. */
    .algo_type = 1,
    .preproc_block_sz = 256,
    .postproc_block_sz = 256,
    /* Model access methods (use eNPU to offload or not) */
    .access_method = MODEL_ENPU_OFFLOADED,
#else
    /* Default algorithm is data passthrough */
    /* Non-offloaded gain model with gain ratio 0.5 */
    .model_file = "gain_sw.eai",
    /* Model access methods (use eNPU to offload or not) */
    .access_method = MODEL_NON_ENPU_OFFLOADED,
#endif // ML_FFT_ALGORITHM

#endif // ML_LOOPBACK

    /* Default operator bundle name: Change this to the actual
    capability
    bundle */
    /* download_ml covers both ml engine and ml base capability
    */
    .operator_bundle = "download_ml.dkcs",
    /* Batch Reset count: Resets the persistent tensors after
    this count
    if not zero */
    .batch_reset_count = 0
};

```

Machine Learning loopback mode

The following figure shows an example diagram from KDAT graph analysis.

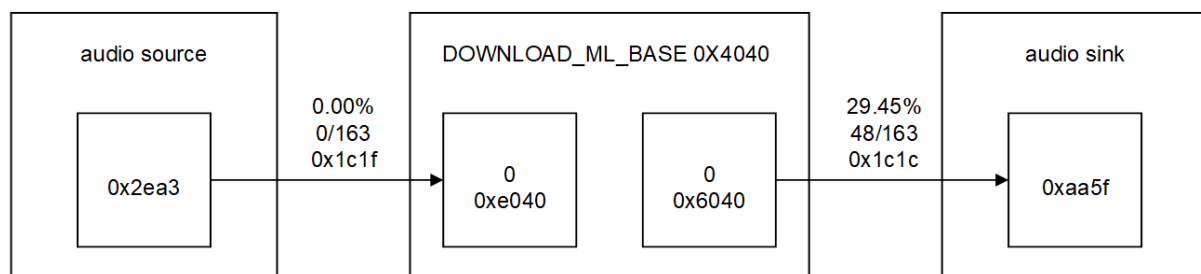


Figure1 Example diagram from KDAT graph analysis

The operator id, endpoint ids, transform ids and the data level in the buffers could change based on the audioSS state when executing the example. The workflow is as follows:

1. Download the ml bundle to audio memory using the capability download manager and create the downloadable ml base operator (`cap_id = 0x40AF`).
2. Create ADC (audio source) and DAC (audio source) endpoints and configure the sampling rate to 16 kHz.
3. Configure `ml_base` by loading the model `gain_sw.eai` to audioSS using `OperatorDataLoadEx()` and use operator messages for loading and activating within the operator context.
4. Connect ADC and DAC to downloadable ml base.
5. Start the operator.

The expected audio output is -6 dB.

Machine Learning loopback with FFT mode

The FFT mode of the application uses a different model, `fc2_hw`. This model can be offloaded to eNPU3 and specified through the operator message to the `ml_base` operator.

To set FFT as the processing algorithm in the downloadable ml base, an operator message is needed to pass algorithm type (0 = data passthrough and 1 = FFT), preprocessing size in bytes and postprocessing size in bytes, where the FFT length is dependent on processing size. For example, in this case, 256 is passed to represent 128 FFT length in the 16-bit model. This step should be before model activation.

Note: The output waveform doubles since the ML model uses a weight diagonal matrix with multiplier 2 in the FC layer.

Machine Learning Engine Self-test mode

The ML engine self-test mode is a stand-alone operator mode for validating a model using raw activation inputs and expected output called golden output.

- It creates a downloadable ML engine (cap_id = 0x40B1) operator from the same ML bundle.
- The Self-Test mode loads three files using `OperatorDataLoadEx()`. In this example, these three files include the model file,

`vww_hw.eai` model, activity input file, `vww_input.raw` and output file, `vww_golden.raw`. The operator is instructed by using an operator message to load the model. Activation is not needed here since self-test mode activates the model internally.

To run the ML Engine Self-test:

- Start the operator.
- Start the self-test mode using the operator message with provided input and golden output handle ids.

Changing the test set (that is, model, input and output files) can validate a different model. The input and output raw files must be aligned with the expected tensor size per inference. Otherwise, the test would not end. If there are multiple tensors for the activity input, it must be sequentialized and converted into a single input file. It is expected to connect KDAT to get the model analysis.

7.3 Debugging Machine Learning application using Kymera Debugging and Analysis Tool

Kymera Debugging and Analysis Tool (KDAT) is a powerful tool to extract audio analysis (debug log, stream, cbops etc.) or model analysis (ml analysis) for the applications, which supports both core dump file and live chip. Self-test mode is used as a vivid example to demonstrate how to get the eNPU3, layer-wise performance and model outputs from a live chip.

KDAT debug log analysis

Command to run in KDAT env, `show_html = True` to display analysis in html format (recommended):

```
mdsp.analyses.debuglog.run_all(show_html=True)
```

It returns the mdsp log below.

Blue rectangular box: vww model basic information (1 input/output tensor, 33 layers etc).

Red rectangular box shows model performance and test result:

- Self-test of one inference is PASS. Overall performance (total cycles, total offloaded layers, total memory) is returned.

- Job profile ticks have three values. From left to right, the first one is eNPU processing time (1013596), interrupt to job complete time (6064) and total eNPU driver execution cycles (1031517). In this case, we only have one offloaded job (merged group id = 1) for vww model, so only one job profile tick is given. For a complicated model, multiple job profile ticks will be displayed here. These multiple cycles should be accumulated to get the total eNPU performance.

```
MLE: Activate usecase 1 with num input tensors=1 and num output tensors=1
MLE: Total input tensor size is 27648 and output size is 2
MLE Self Test: Populated Data
Profiling is enabled correctly!
Number of layers in this model: 33
MLE_ST: The input size is 27648 and processed 0
MLE_ST: The golden size is 2 and processed 0
MLE_ST: Copied 27648 input data to offset 0
MLE ST: Register Self-Kick
MLE : Self test started for UC=1 with 1 inferences, input handle 0x6c02, golden handle 0x6c03
MLE: Total number of frame processed are = 1
ACCMD tx: con 0, cmdid 0x4008, seqno 0x0037, len 6 words
        tx: 0000 4040 001b
Job profile ticks[1013596]:[6064]:[1031517]
MLE ST: Test completed! The golden data match with all data size 2
MLE ST: self-test PASS - total cycles 1121002, total offloaded layers 29 and total memory 262417
ACCMD tx: con 0, cmdid 0x0010, seqno 0x0006, len 14 words
        tx: 4040 0000 0013 0007 443e
        tx: 0000 1aea 0011 001d 0000
        tx: 0000
ACCMD rx: con 0, cmdid 0x4003, seqno 0x0006, len 4 words
        rx: 0000
```

KDAT Machine Learning analysis

ML analysis can give detailed information by using the following commands:

- `show_output = False`: Disables the display of model output (enable for outputs).
- `show_html = True`: Displays analysis in html format (recommended).

```
mdsp.analysises.ml.run_all(show_output=False, show_html=True)
```

Model output analysis using KDAT

The following figure displays the model output (hex format).

ML Usecase 1

eAI Init: Yes
Use eNPU Offloading: Yes

EAI Analysis

Model Basic Information:

Num of Input Tensor: 1
Num of Output Tensor: 1
Input Tensor Size: 27648
Output Tensor Size: 2
Tensor 0 Model Output (Size: 2):

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Address 0x0	0x30	0xfb														

Figure2 Model output analysis using KDAT tool

Offloaded information

- Layer name
- Size of each layer
- If the layer is offloaded
- If multiple offloaded layers are grouped together (layers are grouped into single job for efficient processing). To see the eNPU3 performance, each eNPU3 job tick profiler is associated to each group IDs.
- Offloaded count

offloaded_information.PNG

Profiling information (MDSP + eNPU3)

- Total time (us)
- Layer-wise cycles
- Execution count (* layer is grouped to first layer, execute in a one go)

Profiling:

	Layer	Total Time (us)	Total Cycles	Execution count
Layer 0	SUPERCONV	1	711	1
Layer 1	SUPERCONV	0	0	*1
Layer 2	SUPERCONV	0	0	*1
Layer 3	SUPERCONV	0	0	*1
Layer 4	SUPERCONV	0	0	*1
Layer 5	SUPERCONV	0	0	*1
Layer 6	SUPERCONV	0	0	*1
Layer 7	SUPERCONV	0	0	*1
Layer 8	SUPERCONV	0	0	*1
Layer 9	SUPERCONV	0	0	*1
Layer 10	SUPERCONV	0	0	*1
Layer 11	SUPERCONV	0	0	*1
Layer 12	SUPERCONV	0	0	*1
Layer 13	SUPERCONV	0	0	*1
Layer 14	SUPERCONV	0	0	*1
Layer 15	SUPERCONV	0	0	*1
Layer 16	SUPERCONV	0	0	*1
Layer 17	SUPERCONV	0	0	*1
Layer 18	SUPERCONV	0	0	*1
Layer 19	SUPERCONV	0	0	*1
Layer 20	SUPERCONV	0	0	*1
Layer 21	SUPERCONV	0	0	*1
Layer 22	SUPERCONV	0	0	*1
Layer 23	SUPERCONV	0	0	*1
Layer 24	SUPERCONV	0	0	*1
Layer 25	SUPERCONV	0	0	*1
Layer 26	SUPERCONV	0	0	*1
Layer 27	AVERAGE_POOL	0	0	*1
Layer 28	SUPERADD	2207	1101729	1
Layer 29	NO_OP	1	210	1
Layer 30	SUPERGEMM	23	11363	1
Layer 31	SOFTMAX	7	3484	1
Layer 32	SUPERADD	3	1131	1

Other information

- Memory and overall performance analysis
- eNPU driver analysis (core, client, jobs) → useful for eNPU debugging in case of error.

7.4 Developing a new Machine Learning application

For information about developing a customer application using build, see the *Qualcomm S7 Gen 1 Sound Platform ED2 ADK Software Getting Started Guide*.

From ML perspective, the self-test application can be used for initial model benchmarking, performance analysis and power measurement with given input and golden data.

To test with real audio, use ML base capability source code as a starting point to customize preprocessing and postprocessing module and create a new downloadable ML capability.

Use the ML loopback application as a reference to attach ADC/DAC endpoints to customized ML capability and analyses the output audio waveform.

KDAT can be used for audio and ML model analysis on the hardware.

8 Working example: Creating and deploying Machine Learning model on target audio platform

This example captures the steps to take a ONNX model through the eAI builder and running it on the eNPU3.

8.1 Convert the model to eAI

This step is performed on Linux tools (WSL) located at `/opt/qtil/ ADK_Toolkit_2.0.2.27/eai_toolkit/eai_runtime>`.

Note: You might need to change the permissions on these folders using `sudo chmod -R 777`.

Create the quantization template

Model json template creation:

```
./tools/model_builder/eai_builder --onnx ./sample_models/v3/fixed/
model1/onnx_model/sample_model.onnx --create_quantization_template ./
sample_models/v3/fixed/model1/model/sample_model_quant_template.json
```

Create, visualize, and run the x86 version of the eAI model (golden reference)

1. Create the x86 version of the model.

Model creation:

```
./tools/model_builder/eai_builder --onnx ./sample_models/
v3/fixed/model1/onnx_model/sample_model.onnx --
quantization_config_file ./sample_models/v3/fixed/model1/
model/sample_model_quant_template.json --output ./sample_
models/v3/fixed/model1/onnx_model/sample_model_x86.eai --
target_env adsp --enable_enpu_ver v3
```

2. To visualize the generated x86 version of the eAI model file:

- Launch or open the Netronix-<version>.AppImage located in /tools/model_visualizer/Netronix-<version>.AppImage and open the generated eAI model file or
- Execute the following command in the bash shell by replacing <version> with latest available Netronix version in the toolkit:

```
./tools/model_visualizer/Netronix-<version>.AppImage ./sample_models/v3/fixed/model1/onnx_model/sample_model_x86.eai  
# If the above command fails to run, try:  
./tools/model_visualizer/Netronix-<version>.AppImage ./sample_models/v3/fixed/model1/onnx_model/sample_model_x86.eai --no-sandbox
```

3. Run the model to get the x86 golden reference output.

```
./eai_sample_app/x86/fix32/eai_sample_app -m sample_models/v3/fixed/model1/onnx_model/sample_model_x86.eai -i sample_models/v3/fixed/model1/input/quantized_sample_model_input_fixed.raw -o sample_models/v3/fixed/model1/my_output
```

Create, visualize, and run the model for target (Tensilica + eNPU3)

1. Create the quantization template.
2. Use the template to create a model for the Tensilica + eNPU.

Model creation:

```
./tools/model_builder/eai_builder --onnx ./sample_models/v3/fixed/model1/onnx_model/sample_model.onnx --quantization_config_file ./sample_models/v3/fixed/model1/onnx_model/quantize_config.json --output ./sample_models/v3/fixed/model1/onnx_model/sample_model.eai --target_env tensilica --enable_enpu_ver v3
```

3. To visualize the generated tensilica version of the eAI model file:

- Launch or open Netronix-<version>.AppImage located in /tools/model_visualizer/Netronix-<version>.AppImage and open the generated eAI model file or
- Execute the following command in the bash shell by replacing <version> with latest available Netronix version in the toolkit:


```
./tools/model_visualizer/Netronix-<version>.AppImage ./
sample_models/v3/fixed/model1/onnx_model/sample_model.eai
# If the above command fails to run, try:
./tools/model_visualizer/Netronix-<version>.AppImage ./
sample_models/v3/fixed/model1/onnx_model/sample_model.eai
--no-sandbox
```

4. Run the model to check tensilica output against x86*.

```
cd eai_sample_app/tensilica
./eai_sample_app.sh -m ../../sample_models/v3/fixed/
model1/onnx_model/sample_model.eai -i ../../sample_
models/v3/fixed/model1/input/quantized_sample_model_
input_fixed.raw -o ../../sample_models/v3/fixed/model1/
output/ -g ../../sample_models/v3/fixed/model1/my_output/
output_0.raw
```

8.2 Setup the Xtensa SDK and HiFi package

Note: This is a prerequisite.

To setup the Xtensa SDK and HiFi package:

1. Download Xtensa SDK for Linux (command line) and Snapdragon S7 / S5 Gen 1 Sound Platform HiFi 4 Package for Linux from <https://tensilicatools.com/download/qualcomm-voice-music-sdk-ri-2021-7-version/>.
2. Copy the downloaded files into a directory. For example, /mnt/c/wsl_ubuntu22/xtensa_and_hifi.
3. Run the following commands to install and setup the Xtensa SDK and HiFi packages:

```
# navigate to the directory containing the .tgz files
cd /mnt/c/wsl_ubuntu22/xtensa_and_hifi

# extract the files
tar -xvzf XtensaTools_RI_2021_7_linux.tgz
tar -xvzf HiFi4_Aria2_0_linux_redist.tgz

# installation
cd RI-2021.7-linux/HiFi4_Aria2_0
./install # during installation, when prompted, specify the
tools path, including the root path (e.g., /mnt/c/wsl_ubuntu22/
```

```
xtensa_and_hifi/RI-2021.7-linux) .
```

4. Set the environment variables for HiFisim (Tensilica) by configuring the Xtensa tool path:

```
cd eai_toolkit/eai_runtime/eai_sample_app
source xtensa_tools_setup.sh /mnt/c/wsl_ubuntu22/xtensa_and_hifi/ # path to the directory where Xtensa SDK and HiFi package is present
# Expected output:
# Configuring Xtensa tools path /mnt/c/wsl_ubuntu22/xtensa_and_hifi/RI-2021.7-linux/XtensaTools
# Confirm the environment variables are set by using the env command or echo $PATH.
```

5. Test the installation by calling HiFisim using the command:

```
cd eai_toolkit/eai_runtime/tools/hifisim
./hifisim_qcc722x_qcc732x_audio --help
```

Copy all of the files into one folder (to be used later on the chip). Required files are the eAI model, the input wav, and the reference output file.

```
cp ./sample_models/v3/fixed/model1/onnx_model/sample_model.eai $ADK_ROOT/example_apps/examples/src/ml/sample_model.eai
cp sample_models/v3/fixed/model1/my_output/output_0.raw $ADK_ROOT/example_apps/examples/src/ml/output0.raw
cp ./sample_models/v3/fixed/model1/input/quantized_sample_model_input_fixed.raw $ADK_ROOT/example_apps/examples/src/ml/input.raw
```

8.3 Prepare the Machine Learning application for the board

The ML application is responsible for loading the model framework and also the specific model in the self-test mode. The example application can be made to run the ML application by using a python script to update the c-code. Run this from the examples folder.

Enable the Machine Learning example application

```
py examples_update.py ml
```

1. Update the ml.h header file to enable the self-test mode.

ml.h

```
/*! Enable 1 of the options for how the ml example will run
* 1) ML_LOOPBACK to run normal ml loopback (default)
```

```

* 2) ML_ENGINE_SELF_TEST to run self-test mode */
//#define ML_LOOPBACK
#define ML_ENGINE_SELF_TEST
/*! ML_FFT_ALGORITHM can be used with ML_LOOPBACK to run a ml
loopback with
* FFT implementation */
//#define ML_FFT_ALGORITHM

```

2. Update the header file to include the model file, input and the reference file.

ml.h

```

/* Configuration of the machine learning examples */
static const ml_config_t ml_config = {
    /* ML usecase id starting from 1 */
    .use_case_id = 1,
#ifdef ML_ENGINE_SELF_TEST
    /* Default capability ID of the ML engine capability */
    .capability_id = 0x40B1,
    /* Offloaded visual waking words TinyML model */
    .model_file = "sample_model.eai",
    /* Model access methods (use enpu to offload or not) */
    .access_method = MODEL_ENPU_OFFLOADED,
    /* Run the input tensor once */
    .num_inferences = 10000,
    .input_file = "input.raw",
    .golden_file = "output0.raw",
#endif //ML_ENGINE_SELF_TEST

#ifdef ML_LOOPBACK

```

3. Update the SConscript.py to include these files into the build system.

SConscript.py

```

p_env.Append(FS_FILES_FOR_PROJ=[os.path.join(p_env['ROOT'],
'audio/qcc532x_qcc522x/audioss/kymera/prebuilt_dkcs/libretto_
release/download_passthrough.dkcs'),
    os.path.join(p_env['ROOT'], 'audio/qcc532x_qcc522x/audioss/
kymera/prebuilt_dkcs/libretto_release/download_ml.dkcs'),
    os.path.join(p_env['ROOT'], 'example_apps/examples/src/ml/
sample_model.eai'),
    os.path.join(p_env['ROOT'], 'example_apps/examples/src/ml/
input.raw'),
    os.path.join(p_env['ROOT'], 'example_apps/examples/src/ml/

```

```
output0.raw']]
```

8.4 Erase the board

This command might need to be run twice to erase the board and to start from a known position.

Erase and flash

```
ATK (c:\qtil\ADK_Toolkit_2.0.1.88_x64\) C:\ADK_Projects\Libretto\
qcc532x-rom-1-0_qtil_standard_oem>nvscmd -eud 1 erase
nvscmd, version 4.0.0.35 Release
Copyright (c) 2010-2023 Qualcomm Technologies International, Ltd.
All Rights Reserved.
Qualcomm Technologies International, Ltd. Confidential and
Proprietary.
```

```
Logging is currently not configured. Set QTILEUD_LOG_LEVEL=4 and
QTILEUDSRV_LOG_LEVEL=4
if required for a client process labelled logfile and qtileudsrv.log
in the working directory.
Otherwise refer to the package README for more information.
Erase the SQIF...
```

```
Loading flash loader software to device
Device not ready to accept flash loader, resetting device
```

```
=====
ERROR: Downloading bootloader Failed.
=====
```

```
Failed
```

```
ATK (c:\qtil\ADK_Toolkit_2.0.1.88_x64\) C:\ADK_Projects\Libretto\
qcc532x-rom-1-0_qtil_standard_oem>nvscmd -eud 1 erase
nvscmd, version 4.0.0.35 Release
Copyright (c) 2010-2023 Qualcomm Technologies International, Ltd.
All Rights Reserved.
Qualcomm Technologies International, Ltd. Confidential and
Proprietary.
```

```
Logging is currently not configured. Set QTILEUD_LOG_LEVEL=4 and
QTILEUDSRV_LOG_LEVEL=4
```

```
if required for a client process labelled logfile and qtileudsrv.log
in the working directory.
Otherwise refer to the package README for more information.
Erase the SQIF...
Loading flash loader software to device
Processing chunk 0
Processing chunk 1
Run flashloader
Running
Switching to USBDBG 1
Success
```

Note: This command may need to be run twice.

8.5 Build and burn the image to the device

1. Run the following command to build the example image:

Build command

```
vbuild QCC5228-AA_DEV-BRD-AA_EXAMPLES -t single_image
```

2. Program the chip with the example application using the command:

Build command

```
nvscmd -eud 1 burn example_apps\examples\workspace\
QCC5228-AA_DEV-BRD-AA\output\examples.hex
```

8.6 Connect KDAT to the device

Connect to the device using the Kymera development and analysis tool.

Start KDAT

```
ATK (c:\qtil\ADK_Toolkit_2.0.2.25_x64\) C:\ADK_Projects\Libretto\
qcc532x-rom-1-0_qtil_standard_oem>kdat -t pydbg:jlink:jtag --target
cda2 -e ./tools/FLAT_Files -b ".\audio\qcc532x_qcc522x\audioss\
kymera\output\libretto_release\build\debugbin\kymera_libretto_audio.
elf"
```

9 Revision history

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyungchul.won@lge.com

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyungchul.won@lge.com

Revision	Date	Description
AA	March 2023	Initial release.
AB	June 2023	- Updated Kymera Machine Learning capability. - Added Kymera Machine Learning example application.
AC	June 2023	Updated the document title.
AD	June 2023	Added Working example and updated Offloading operations using embedded Neural Processing Unit and Kymera Machine Learning example application.
AE	October 2023	- Updated Setting up ML example application. - Editorial updates.
AF	December 2023	Editorial updates.
AG	March 2024	Updated the ML Workflow and eAI tools for model conversion.
AH	November 2025	Updated sections: - Offloading operations using embedded Neural Processing Unit - Operator messages - Kymera Simulation Environment validation Newly added sections: - eAI supported operations and eNPU3 offload - eAI design guidelines - Setup the XtenSA SDK and HiFi package

10 Document references

Document	Reference
Kymera Capability Development User Guide	80-CG107-1
Qualcomm Kymera Simulation Environment (KSL) User Guide	80-62884-1
eNPU3 Design Guidelines	80-37209-1
Qualcomm S7 Gen 1 Sound Platform ED2 ADK Software Getting Started Guide	80-62968-1

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-12 06:35:39 GMT
hyunchul.won@lge.com

11 Terms and definitions

AIMET

AI Model Efficiency Toolkit

eNPU3

Embedded Neural Processing Unit

HiFiSim

HiFi4 Simulator for S7 Gen 1 and S7 Pro Gen 1 sound platforms

KDAT

Kymera Debugging and Analysis Tool is a Python-based debugging and analysis tool for both Qualcomm Kymera Capability and Framework code

KSL

Kymera Simulation Environment

ML

Machine Learning

QTI

Qualcomm International Technologies, Ltd.

Qualcomm MDE

Multicore Development Environment

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("Qualcomm Technologies"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "Qualcomm Internal Use Only", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

THE DOCUMENTATION ACCOMPANYING THE MATERIALS AND/OR RELEVANT PRODUCTS AND SERVICE OFFERINGS MAY INCLUDE IMPORTANT USE LIMITATIONS. ANY DEVIATIONS FROM APPLICABLE USE LIMITATIONS MAY ADVERSELY IMPACT PERFORMANCE, DURABILITY, QUALITY OR SAFETY. YOU ASSUME ALL RISKS AND LIABILITIES ASSOCIATED WITH ANY DEVIATIONS.